



TECNICATURA SUPERIOR EN

**Ciencia de Datos e  
Inteligencia Artificial**

## **TÉCNICAS DE PROCESAMIENTO DEL HABLA**

EJE N°1: Fundamentos y Procesamiento Digital de la Señal

---

### **LABORATORIO N°2**

**Tema: Preparación de Datos Crudos de  
Audio en Python**

## Laboratorio 2: Preparación de Datos Crudos de Audio en Python

**Entorno:** Google Colab | **Nivel:** Introductorio | **Duración:** 90-120 minutos

**Instrucciones:** Ejecuta cada celda en orden. Los comentarios guían cada paso. Se incluye generación de audio sintético para que no necesites archivos externos.

### 1. Configuración Inicial

python

```
# 📦 Instalación de dependencias (ejecutar una vez)
!pip install librosa numpy matplotlib scipy ipywidgets

# 📦 Importación de módulos
import numpy as np
import matplotlib.pyplot as plt
import librosa
import librosa.display
from scipy import signal
from IPython.display import Audio, display
import ipywidgets as widgets
from ipywidgets import interact

# ⚙️ Configuración de visualización
plt.rcParams['figure.figsize'] = (10, 4)
plt.rcParams['font.size'] = 11
plt.style.use('seaborn-v0_8-whitegrid')
print("✅ Entorno listo. ¡Comencemos!")
```

## 2. Parte 1: Generación de Audio Sintético (Para Práctica)

Como no todos tienen archivos de audio a mano, crearemos señales de prueba.

python

```
def generar_audio_prueba(frecuencias, duracion=3.0, sr=22050, ruido=0.02):
    """
    Genera una señal de audio sintética para prácticas.

    Parámetros:
    - frecuencias: lista de frecuencias en Hz (ej. [200, 400])
    - duracion: duración en segundos
    - sr: sample rate
    - ruido: nivel de ruido blanco añadido (0.0 = limpio)
    """
    t = np.linspace(0, duracion, int(sr * duracion), endpoint=False)
    señal = np.zeros_like(t)

    # Sumar ondas senoidales
    for f in frecuencias:
        señal += np.sin(2 * np.pi * f * t)

    # Normalizar a [-0.8, +0.8] para dejar margen
    señal = 0.8 * señal / np.max(np.abs(señal))

    # Añadir ruido blanco opcional
    if ruido > 0:
        señal += ruido * np.random.randn(len(señal))
        señal = np.clip(señal, -1, 1) # Evitar saturación

    return señal, sr

# 🎵 Crear 3 ejemplos para el laboratorio
audio_limpio, sr = generar_audio_prueba([300], ruido=0.0)
audio_ruidoso, _ = generar_audio_prueba([300], ruido=0.05)
audio_largo, _ = generar_audio_prueba([200, 500], duracion=8.0, ruido=0.01)

print(f"✓ Generados: limpio ({len(audio_limpio)}/sr:.1f)s), ruidoso, largo ({len(audio_largo)}/sr:.1f)s")
```

### 3. Parte 2: Carga y Diagnóstico Visual

python

```
def diagnosticar_audio(audio, sr, titulo="Diagnóstico"):
    """Muestra waveform y espectrograma básico para evaluación visual"""

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6))

    # Waveform
    librosa.display.waveshow(audio, sr=sr, ax=ax1, color='steelblue')
    ax1.set_title(f'{titulo} - Waveform')
    ax1.set_xlabel('Tiempo (s)')
    ax1.set_ylabel('Amplitud')
    ax1.grid(True, alpha=0.3)

    # Espectrograma (STFT básica)
    D = np.abs(librosa.stft(audio))
    S_db = librosa.amplitude_to_db(D, ref=np.max)
    img = librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis='hz', ax=ax2,
    cmap='magma')
    ax2.set_title('Espectrograma (dB)')
    ax2.set_xlabel('Tiempo (s)')
    ax2.set_ylabel('Frecuencia (Hz)')
    plt.colorbar(img, ax=ax2, format='%+2.0f dB')

    plt.tight_layout()
    plt.show()

    # Métricas rápidas
    print(f"🕒 Duración: {len(audio)/sr:.2f} s")
    print(f"🔊 Amplitud: [{audio.min():.3f}, {audio.max():.3f}]")
    print(f"⚡ Energía RMS: {np.sqrt(np.mean(audio**2)):.4f}")
    print(f"🌀 Zero-Crossing Rate medio:
    {librosa.feature.zero_crossing_rate(audio).mean():.4f}")

    # 🔍 Diagnosticar los audios de prueba
    print("📁 Audio limpio:")
    diagnosticar_audio(audio_limpio, sr, "Audio Limpio (300 Hz)")

    print("\n📁 Audio ruidoso:")
    diagnosticar_audio(audio_ruidoso, sr, "Audio con Ruido")
```

#### 4. Parte 3: Limpieza Interactiva (Trimming y Suavizado)

python

```
def aplicar_limpieza(audio, sr, top_db, clip_limit=0.95):
    """Aplica trimming y clipping suave, muestra comparación"""

    # 1. Recortar silencios
    audio_trim, _ = librosa.effects.trim(audio, top_db=top_db)

    # 2. Suavizar picos extremos (evitar saturación digital)
    audio_clean = np.clip(audio_trim, -clip_limit, clip_limit)

    # Comparación visual
    plt.figure(figsize=(10, 5))
    plt.subplot(2, 1, 1)
    librosa.display.waveshow(audio, sr=sr, label='Original', alpha=0.6)
    plt.title(f'Antes de limpieza (top_db={top_db} dB)')
    plt.ylabel('Amplitud')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.subplot(2, 1, 2)
    librosa.display.waveshow(audio_clean, sr=sr, color='green', label='Limpio')
    plt.title('Después de trimming + clipping suave')
    plt.xlabel('Tiempo (s)')
    plt.ylabel('Amplitud')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    # Resumen
    reduccion = (1 - len(audio_clean)/len(audio)) * 100
    print(f"✓ Silencios recortados: {reduccion:.1f}% del audio original")
    print(f"✓ Amplitud acotada a [-{clip_limit}, +{clip_limit}]")

    return audio_clean

# 🎧 Control interactivo para experimentar con top_db
print("🎧 Ajusta el umbral de trimming (10-60 dB) y observa el efecto:")
interact(
    lambda db: aplicar_limpieza(audio_ruidoso, sr, db),
```

```
db=widgets.IntSlider(min=10, max=60, step=5, value=30, description='top_db  
(dB):')  
);
```

## 5. Parte 4: Normalización Completa (Pipeline)

python

```
def pipeline_normalizacion(audio, sr, target_sr=22050, target_dur=3.0,
normalize_peak=True):
    """
    Pipeline completo de normalización:
    1. Mono (si es estéreo)
    2. Remuestreo
    3. Normalización de amplitud
    4. Ajuste de duración
    """

    # Paso 1: Asegurar mono
    if audio.ndim == 2:
        audio = librosa.to_mono(audio)
        print("🔊 Convertido a mono")

    # Paso 2: Remuestrear si es necesario
    if sr != target_sr:
        audio = librosa.resample(audio, orig_sr=sr, target_sr=target_sr)
        sr = target_sr
        print(f"🔊 Remuestreado a {target_sr} Hz")

    # Paso 3: Normalizar amplitud (peak normalization)
    if normalize_peak and np.max(np.abs(audio)) > 0:
        audio = audio / np.max(np.abs(audio))
        print("✅ Amplitud normalizada a [-1, +1]")

    # Paso 4: Ajustar duración (pad o crop)
    samples_objetivo = int(target_dur * sr)
    if len(audio) > samples_objetivo:
        # Crop: tomar el centro para no perder inicio/fin abruptamente
        inicio = (len(audio) - samples_objetivo) // 2
        audio = audio[inicio:inicio + samples_objetivo]
        print(f"✂️ Recortado a {target_dur}s (centrado)")
    else:
        # Pad: rellenar con silencios al final
        audio = np.pad(audio, (0, samples_objetivo - len(audio)), mode='constant')
        print(f"■ Rellenado con silencios a {target_dur}s")

    return audio, sr

# 🔊 Aplicar pipeline al audio largo
```

```
print("🔊 Aplicando pipeline de normalización al audio de 8s...")
audio_norm, sr_norm = pipeline_normalizacion(audio_largo, sr, target_dur=3.0)

# Visualizar resultado
diagnosticar_audio(audio_norm, sr_norm, "Audio Normalizado (3.0s, 22050 Hz)")

# 🔊 Escuchar (opcional)
print("\n🔊 Reproduciendo audio normalizado...")
display(Audio(audio_norm, rate=sr_norm))
```



## 6. Parte 5: Extracción de Features Básicas para EDA

python

```
def extraer_features_basicas(audio, sr):
    """Extrae métricas descriptivas simples para análisis exploratorio"""

    features = {}

    # Energía RMS (potencia promedio)
    rms = librosa.feature.rms(y=audio)[0]
    features['rms_mean'] = np.mean(rms)
    features['rms_std'] = np.std(rms)

    # Zero-Crossing Rate (textura de la señal)
    zcr = librosa.feature.zero_crossing_rate(audio)[0]
    features['zcr_mean'] = np.mean(zcr)
    features['zcr_std'] = np.std(zcr)

    # Centroide espectral (brillo aproximado)
    centroid = librosa.feature.spectral_centroid(y=audio, sr=sr)[0]
    features['centroid_mean'] = np.mean(centroid)

    # Roll-off (frecuencia que acumula 85% de energía)
    rolloff = librosa.feature.spectral_rolloff(y=audio, sr=sr)[0]
    features['rolloff_mean'] = np.mean(rolloff)

    # Duración y sample rate
    features['duration'] = len(audio) / sr
    features['sample_rate'] = sr

    return features

# 📁 Extraer y comparar features de los 3 audios
print("📁 Comparación de features básicas:\n")
print(f'{"Feature":<20} | {"Limpio":>10} | {"Ruidoso":>10} | {"Normalizado":>12}')
print("-" * 60)

for nombre, aud in [("Limpio", audio_limpio), ("Ruidoso", audio_ruidoso), ("Norm.",
audio_norm)]:
    f = extraer_features_basicas(aud, sr if nombre!="Norm." else sr_norm)
    print(f'RMS mean      | {f["rms_mean"]:>10.4f} |
{extraer_features_basicas(audio_ruidoso, sr)["rms_mean"]:>10.4f} |
{f["rms_mean"]:>12.4f}')
```

```
print(f'ZCR mean      | {f['zcr_mean']:>10.4f} |  
{extraer_features_basicas(audio_ruidoso, sr)['zcr_mean']:>10.4f} |  
{f['zcr_mean']:>12.4f}")  
print(f'Centroid (Hz) | {f['centroid_mean']:>10.1f} |  
{extraer_features_basicas(audio_ruidoso, sr)['centroid_mean']:>10.1f} |  
{f['centroid_mean']:>12.1f}")  
print("-" * 60)  
break # Solo mostrar una fila completa para no saturar  
  
print("\n💡 Observa cómo el ruido afecta ZCR y RMS, mientras que el centroid se  
mantiene estable si la frecuencia principal no cambia.")
```

## 7. Parte 6: Exportación y Estructura del Dataset

```
def exportar_audio_preparado(audio, sr, nombre_archivo, ruta="."):
    """Exporta audio procesado en formato WAV sin compresión"""
    from scipy.io import wavfile

    # Escalar a int16 para WAV estándar (rango [-32768, 32767])
    audio_int16 = np.int16(audio * 32767)

    # Guardar
    ruta_completa = f"{ruta}/{nombre_archivo}.wav"
    wavfile.write(ruta_completa, sr, audio_int16)

    print(f"✓ Exportado: {ruta_completa}")
    print(f" • Sample rate: {sr} Hz")
    print(f" • Muestras: {len(audio)}")
    print(f" • Formato: WAV 16-bit PCM")

    return ruta_completa

# 📁 Exportar el audio normalizado
exportar_audio_preparado(audio_norm, sr_norm, "ejemplo_01_preparado")

# 📁 Crear estructura de metadatos (simulada)
import pandas as pd

metadatos = pd.DataFrame([
    'id': 'ejemplo_01',
    'archivo_original': 'sintetico_largo.wav',
    'duracion_final': len(audio_norm)/sr_norm,
    'sample_rate': sr_norm,
    'rms_mean': extraer_features_basicas(audio_norm, sr_norm)['rms_mean'],
    'zcr_mean': extraer_features_basicas(audio_norm, sr_norm)['zcr_mean'],
    'centroid_mean': extraer_features_basicas(audio_norm,
sr_norm)['centroid_mean'],
    'preprocesado': 'trim+norm+resample+pad',
    'fecha': '2026-04-12'
])

print("\n📁 Metadatos generados:")
print(metadatos.to_string(index=False))

# Guardar CSV (en Colab, se descarga al ejecutar)
# metadatos.to_csv("metadatos_dataset.csv", index=False)
```

## 8. Actividades para el Estudiante

Responda en un documento o celda de markdown:

1. **Diagnóstico:** Al visualizar un waveform, ¿qué patrones te harían sospechar que un audio necesita trimming? ¿Y qué indicaría saturación (clipping)?
2. **Normalización:** Si tienes un dataset con audios de 1s a 10s, y tu modelo requiere 2.5s fijos, ¿qué estrategia elegirías para los audios cortos (<2.5s) y por qué? ¿Y para los largos?
3. **Features:** Compara los valores de zcr\_mean entre el audio limpio y el ruidoso. ¿Por qué el ruido blanco aumenta la Zero-Crossing Rate? ¿En qué tipo de tarea de clasificación podría ser útil esta feature?
4. **Pipeline:** Diseña una función que aplique preprocessing condicional: si el audio ya está a 22050 Hz, no remuestrear; si su duración está entre 2.8s y 3.2s, no ajustar. ¿Qué ventajas tiene este enfoque "inteligente"?

### Checklist de cierre del Laboratorio 2:

- ☐ Puedo cargar y diagnosticar un audio con librosa
- ☐ Sé aplicar trimming y normalización de forma interactiva
- ☐ Entiendo el pipeline completo de preparación
- ☐ Puedo extraer features básicas para EDA
- ☐ Sé exportar datos procesados con metadatos

**¿Listo para el Tema 3?** En el próximo módulo aplicaremos la Transformada de Fourier para análisis espectral y comenzaremos a "ver" el contenido frecuencial de los audios preparados.